

A randomized neural network based Petrov–Galerkin method for approximating the solution of fractional order boundary value problems

John P. Roop

Department of Mathematics & Statistics, North Carolina A & T State University, Greensboro, NC 27411, USA

ARTICLE INFO

Keywords:

Fractional derivative
Petrov–Galerkin method
Randomized neural network

ABSTRACT

This article presents the implementation of a randomized neural network (RNN) approach in approximating the solution of fractional order boundary value problems using a Petrov–Galerkin framework with Lagrange basis test functions. Traditional methods, like Physics Informed Neural Networks (PINNs), use standard deep learning techniques, which suffer from a computational bottleneck. In contrast, RNNs offer an alternative by employing a random structure with random coefficients, only solving for the output layer. We allow for the application of numerical analysis principles by using RNNs as trial functions and piecewise Lagrange polynomials as test functions. The article covers the construction and properties of the RNN basis, the definition and solution of fractional boundary value problems, and the implementation of the RNN Petrov–Galerkin method. We derive the stiffness matrix and solve it using least squares. Error analysis shows that the method meets the requirements of the Lax–Milgram lemma along with a Céa inequality, ensuring optimal error estimates, depending on the regularity of the exact solution. Computational experiments demonstrate the method's efficacy, including multiples cases with both regular and irregular solutions. The results highlight the utility of RNN-based Petrov–Galerkin methods in solving fractional differential equations with experimental convergence.

1. Introduction

In this article, we discuss and implement the use of randomized neural networks (RNN) to approximate the solution to fractional order boundary value problems in a Petrov–Galerkin framework testing against Lagrange basis functions. Using neural networks to solve differential equations is a trending topic, and the bottleneck associated with training neural networks is now being addressed in a number of ways. One of the first attempts to use neural networks in solving partial differential equations is the DGM method as presented in [1]. In this method a deep neural network is set up which approximates the coefficients of the solution. Then a cost functional is set up and a gradient descent algorithm is derived in order to optimize the coefficients within the network. In the case of traditional feed forward neural network the machine learning algorithm may converge very slowly leading to a potential bottleneck in the computational speed and accuracy.

A good explanation of Physics Informed Neural Networks (PINNs) may be found in the review article [2]. In this formulation, a multi-layer feed forward neural network is constructed in which the input arguments of the neural network are the independent variables (including time) and the output arguments of the neural network are the state variables (velocity, pressure, and temperature for Boussinesq flows). Then the resulting state variables are differentiated using finite differences or automatic differentiation and

E-mail address: jproop@ncat.edu.

<https://doi.org/10.1016/j.rinam.2024.100493>

Received 21 July 2024; Received in revised form 25 August 2024; Accepted 29 August 2024

Available online 5 September 2024

2590-0374/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

substituted into the governing equations and weighted initial and boundary conditions. The equation describing the error in the PINN is called the loss function. The machine learning algorithm is based on gradient descent and seeks to minimize the loss function. As we can obtain very fast convergence with respect to our grid parameter h , the PINNs currently require millions of samples in the training set, trained for hours on GPU architecture, and still can only obtain at best 1% relative error for the most complicated situations. Possible solutions to the bottleneck of training PINNs are given in [3], in which the authors decompose the loss function into a linear and nonlinear component and use stacked neural networks which give the PINN a multigrid feel. These decomposition techniques increase the accuracy and running time for PINNs. A PINN based solver for fractional diffusion equations was given in [4], in which the loss function is based on a fractional differential equation. Other recent articles using PINN with fractional order PDEs include [5,6].

Unlike PINNs, there is another type of neural network that can be applied to numerical analysis problems, RNN [7]. Another name for RNN is the Extreme Learning Machine [8]. Throughout the course of this article, we simply refer to the model in question as the RNN. As opposed to a traditional feed-forward neural network the RNN creates a random structure with random coefficients and it is only in the output layer where the coefficients are modeled using a carefully constructed analytical algorithm. The advantages of this approach are two-fold. First, all the issues that arise in potentially slow convergence of the gradient descent algorithm are avoided. Second, because of the analytical nature of the algorithms to find the coefficients, typically using standard least squares techniques, numerical analysis principles may be successfully applied and analysis may be performed.

Y. Shang and collaborators have used randomized neural networks in a Petrov–Galerkin framework [9]. In a recent series of articles J. Sun and collaborators have proposed using RNN as basis functions in a discontinuous Galerkin setting [10,11]. In this framework, the domain is chopped up into finite subdomains and the discontinuous Galerkin method is utilized to model the solution of the ordinary or partial differential equation. In this article, we propose a similar approach which uses a Petrov–Galerkin framework. We do not implement a discontinuous Galerkin method. Instead we choose to model the solution of a fractional boundary value problem as a single RNN with randomly chosen coefficients and the coefficients of the output layer are modeled using a Petrov–Galerkin method where the trial functions form the coefficients of the RNN and test functions are piecewise Lagrange polynomials as is typically found in the Galerkin finite element approach.

Articles discussing fractional differential operators have been featured in *Physics Today* [12], and *Nature* [13]. Solutes moving through aquifers [14] and soil and sand columns [15] often deviate from a Fickian, second-order, governing equation because of large deviations from the stochastic process of Brownian motion. Nonlocal models have received much attention due to their application in problems such as phase transition [16,17], chaotic dynamics [18], and viscoelasticity [19]. Researchers have successfully developed finite difference [20,21] and finite element methods for solving fractional diffusion equations [22,23]. This is the first effort in applying the RNN Petrov–Galerkin concept to fractional differential equations. In this article we construct a random neural network RNN suitable for function approximation. Then we use the RNN basis functions as the trial functions and the standard linear continuous tent functions as the test function in a Petrov–Galerkin finite element approximation, where the entries of an underdetermined linear system are given by integrating the bilinear form and the solution is obtained and analyzed in terms of least squares.

The outline of the article is as follows. In Section 2, we give a description of the RNN, the RNN function, RNN basis, and results pertaining to the approximation properties of the RNN. In Section 3, we give the definition of the fractional diffusion operator(s) in one spatial dimension, and results pertaining to the variational solution of the two-point fractional boundary value problem. In Section 4, we describe the RNN Petrov–Galerkin method. Finally, in Section 5 we give the results of six computational experiments where we have implemented the method in Matlab.

2. The randomized neural network basis

In this section we give the definition and properties of the RNN function, the RNN basis and the properties thereof. The randomized neural network can be designed in any way possible. However, for the purposes of this article, we consider a rather simple straightforward feedforward network with randomized internal coefficients. We traditionally associate the lower case Roman letters u and v with the trial and test functions in a finite element method, so we have used as our RNN coefficient labels the upper case Roman letters $\tilde{U}$, $\tilde{V}$, and W . In this case $\tilde{U}$ is a randomly chosen vector, $\tilde{V}$ is a randomly chosen square matrix, and W is a vector which gives the weights on the output node. A visual depiction of the RNN function used in this article is in Fig. 1. As this not a decision or classification neural network, notice how the design allows for modeling of a function over a given interval.

Definition 1 (RNN Function). Let $W \in \mathbb{R}^N$, $\tilde{V} \in \mathbb{R}^{N \times N}$, $\tilde{U} \in \mathbb{R}^n$, and σ be a sigmoid function. Then the RNN function on the interval (a, b) is found by the relation

$$y(x) = \sum_{j=1}^N W_j \sigma \left(\sum_{i=1}^N \tilde{V}_{i,j} \sigma(\tilde{U}_i x) \right). \quad (1)$$

In the case of a function approximation we consider the $\tilde{V}_{i,j}$ and $\tilde{U}_i$ to be given random real numbers and W_i are considered to be the modeling weights.

Definition 2 (RNN Basis). Let $\tilde{V} \in \mathbb{R}^{N \times N}$ and $\tilde{U} \in \mathbb{R}^N$ be given random weights and σ a sigmoid function. Then the RNN basis associated with that set of weights is

$$\phi_j = \sigma \left(\sum_{i=1}^N \tilde{V}_{i,j} \sigma(\tilde{U}_i x) \right). \quad (2)$$

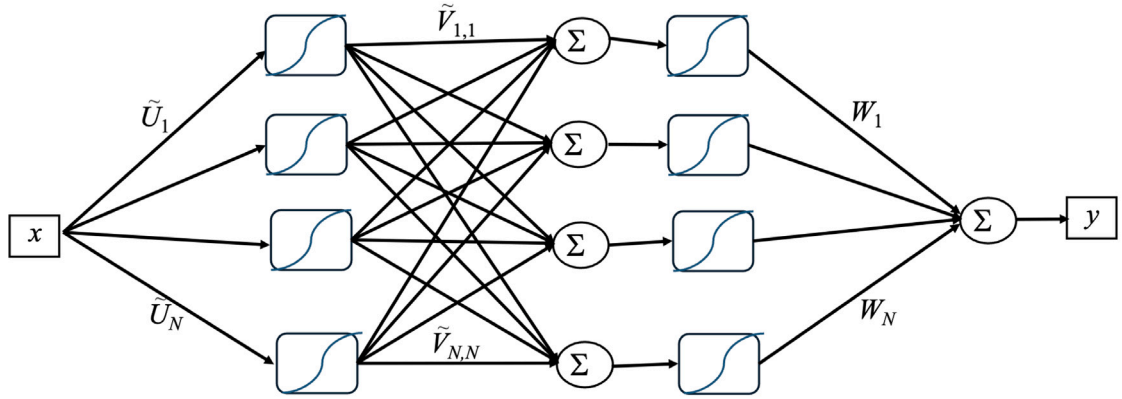


Fig. 1. Visualization of the network basis. $\tilde{U}$, $\tilde{V}$ and W are the coefficients and the boxes with the elongated S symbol represent σ the sigmoid or squasher function.

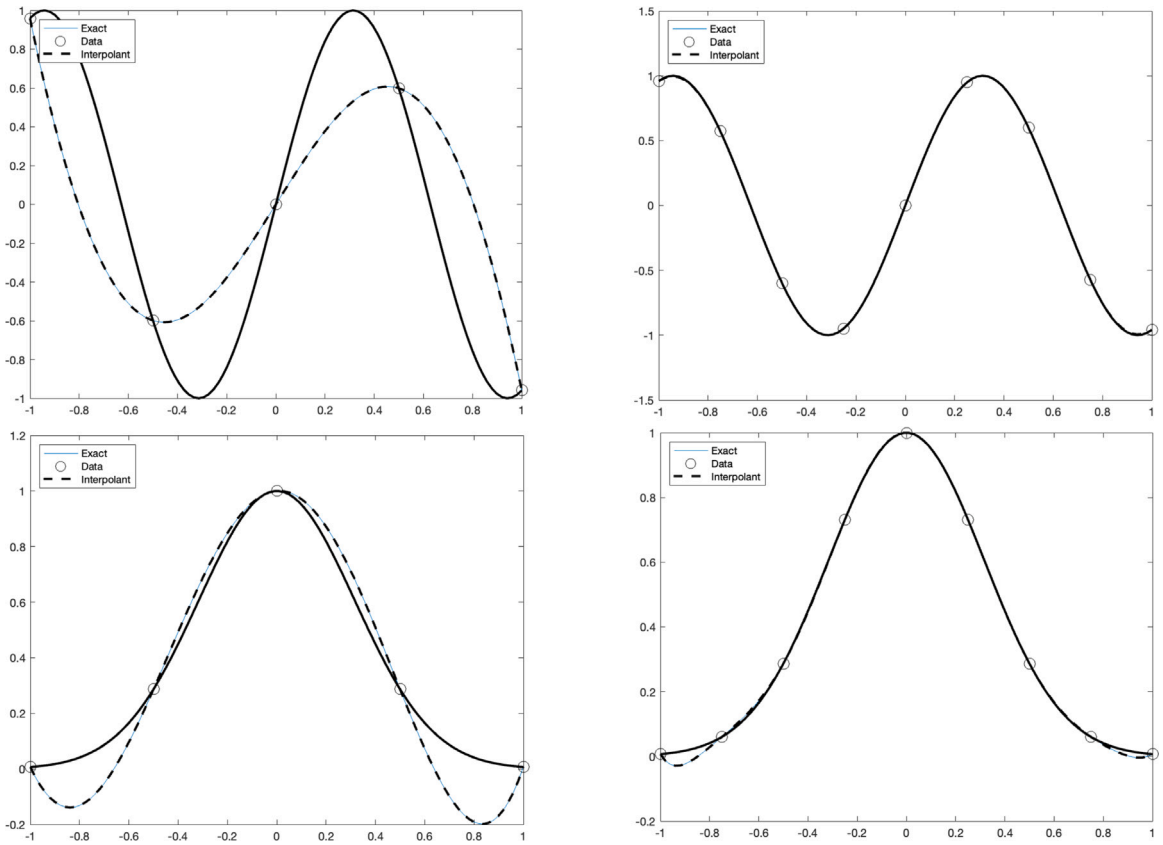


Fig. 2. Left: Approximation of a smooth function (top g_1 , bottom g_2) with RNN and four sample points. Right: Approximation of a smooth function (top g_1 , bottom g_2) with RNN and eight sample points.

Therefore we can consider the following interpolation problem:

Problem 1 (RNN Interpolation Problem). Let $1 \leq M \leq N$, $P_i = (x_i, y_i)$, $i = 1, \dots, M$ be given data points, find the unique RNN function that passes through the data. In this problem the weights $\{W_j\}_{j=1}^N$ solve the $M \times N$ underdetermined linear system

$$A^{RNN} W = \bar{y},$$

where $A_{i,j}^{RNN} := \phi_j(x_i)$.

An assumption is made in [10] that the RNN basis functions are linearly independent which is most likely but the rank of matrix degrades for N large.

In order to illustrate the interpolation properties of the RNN basis model, we choose M sample points on $[-1, 1]$ and use least-squares to interpolate an RNN function with $N = 4M$ internal nodes for each of the functions $g_1(x) = \sin(5x)$ and $g_2(x) = \exp(-5x^2)$. Fig. 2 gives the initial data points, the RNN interpolant, and the exact function on the interval $[-1, 1]$. We can see that as the number of data points increase, the quality of the approximating RNN increases.

Approximation theory involving RNNs proves elusive. The greater scientific community takes as axiomatic an assumption that given the proper learning algorithm, ODEs and PDEs can be solved to machine precision using the right learning algorithm. Because of the way RNNs are trained using least-squares it is possible at least computationally to assess the potential accuracy of RNN models. In [24], the authors give an approximation property regarding the number of linear sub-functions that can be approximated by a neural network. In the article, [25] gives approximation properties of randomized neural networks in terms of the expected value. The paper [10] bases their approximation properties based on a series of assumptions, that a domain decomposition of RNNs mimics the approximation properties and scaling of a typical discontinuous Galerkin finite element Lagrange basis.

3. The fractional boundary value problem

In this section, we review the definitions of the left and right fractional differential operators, define the variational form of the boundary value problem, and give pertinent results to the solution of the variational problem.

Definition 3 (Left Riemann–Liouville Fractional Integral). Let u be defined on the interval (a, b) and $\beta > 0$. Then the *left Riemann–Liouville fractional integral of order β* is defined to be

$${}_a D_x^{-\beta}(u(x)) := \int_a^x \frac{(x - \xi)^{\beta-1}}{\Gamma(\beta)} u(\xi) d\xi. \quad (3)$$

Definition 4 (Right Riemann–Liouville Fractional Integral). Let u be defined on the interval (a, b) and $\beta > 0$. Then the *right Riemann–Liouville fractional integral of order β* is defined to be

$${}_x D_b^{-\beta}(u(x)) := \int_x^b \frac{(\xi - x)^{\beta-1}}{\Gamma(\beta)} u(\xi) d\xi. \quad (4)$$

From hereon we will use as the notations for the left and right fractional integral

$$D^{-\beta}u(x) := {}_0 D_x^{-\beta}(u(x)); \quad D^{-\beta*}u(x) := {}_0 D_x^{-\beta*}(u(x)).$$

Additionally the bold-face fractional derivatives will be utilized to denote the left and right fractional diffusion operators:

$$\begin{aligned} \mathbf{D}^\alpha u(x) &:= D D^{-\beta} Du(x) = D D^{2-\alpha} Du(x); \\ \mathbf{D}^{\alpha*} u(x) &:= D D^{-\beta*} Du(x) = D D^{(2-\alpha)*} Du(x); \end{aligned}$$

where $\beta = 2 - \alpha$.

Problem 2 (Steady State Fractional Diffusion Equation). Given $\Omega = (0, 1)$, $f : \overline{\Omega} \rightarrow \mathbb{R}$, find $u : \overline{\Omega} \rightarrow \mathbb{R}$ such that

$$\begin{aligned} p \mathbf{D}^\alpha u + (1 - p) \mathbf{D}^{\alpha*} u &= f, \quad \text{in } \Omega \\ u &= 0, \quad \text{on } \partial\Omega. \end{aligned} \quad (5)$$

In the following analysis, we denote $(\cdot, \cdot)$ the usual L^2 inner product $(u, v) = \int_0^1 u v dx$, $\|\cdot\|_{L^2}$ the usual L^2 norm and $\|\cdot\|_{H^s}$, $|\cdot|_{H^s}$ the usual fractional (or integer) Sobolev norm and semi-norm respectively.

As in [23], we define the bilinear form $B : H^{\alpha/2} \times H^{\alpha/2} \rightarrow \mathbb{R}$ as

$$B(u, v) := p (D^{-\beta} Du, Dv) + (1 - p) (D^{-\beta*} Du, Dv). \quad (6)$$

We have the variational solution of Problem 2 satisfies the weak formulation as follows.

Problem 3 (Variational Solution). A function $u \in H^{\alpha/2}$ is a variational solution of Problem 2 provided that

$$B(u, v) = (f, v), \quad \forall v \in H_0^{\alpha/2}.$$

The main results in the variational solution are that the variational form satisfies the hypotheses of the Lax–Milgram lemma. That is we have that the bilinear form is continuous over $H^{\alpha/2}$,

$$B(u, v) \leq C_c \|u\|_{H^{\alpha/2}} \|v\|_{H^{\alpha/2}}, \quad (7)$$

and also the bilinear form is coercive.

$$B(u, u) \geq C_o \|u\|_{H^{\alpha/2}}^2. \quad (8)$$

Because $\alpha > 1$, this implies that $\alpha/2 > 1/2$, and thus a solution to the weak formulation satisfies (5) pointwise and vice versa.

4. The RNN Petrov–Galerkin method

In this section, we describe the RNN Petrov–Galerkin method in which the RNN basis functions are used as the trial space and the piecewise Lagrange interpolants are used as the test space. The finite element approximation satisfies the bilinear form over the piecewise Lagrange interpolants and the solution is found through the assembly and solving of a stiffness matrix.

We can readily obtain the format of the fractional derivatives of the piecewise linear polynomials as in [23] or the piecewise quadratic polynomials as in [26]. This leads to a stiffness matrix that is fully occupied, or dense. We can also find the first derivative of each RNN basis function using the chain rule. However, it is not immediately clear how to efficiently and clearly find the fractional derivative of a neural network. Because of this we use the adjoint property of fractional integrals

$$(D^{-\beta}u, v) = (u, D^{-\beta*}v); (D^{-\beta*}u, v) = (u, D^{-\beta}v);$$

to define a second bilinear form which we call $\tilde{B}$ as follows:

$$\tilde{B}(u, v) := p(Du, D^{-\beta*}Dv) + (1-p)(Du, D^{-\beta}Dv). \quad (9)$$

The altered bilinear form $\tilde{B}$ is mathematically and numerically equivalent to (6) except it enables us to maximize the efficiency of the ordinary derivative for the neural network and the fractional derivative of the piecewise Lagrange basis functions.

From before we use $\{\phi_j\}_{j=1}^N$ as the RNN basis, and we define $\{\psi_i\}_{i=1}^n$ as the piecewise Lagrange polynomial basis, we use X_h to denote the span of all the piecewise Lagrange polynomials, the usual test space for a finite element problem.

Problem 4 (RNN Petrov–Galerkin Method). The function $u^{RPG} = \sum_{j=1}^N W_j \phi_j$ is the RNN Petrov–Galerkin solution provided it solves

$$\tilde{B}(u^{RPG}, v) = (f, v), \quad \forall v \in X_h. \quad (10)$$

Numerically we obtain the coefficients W_j through the solution of the matrix equation

$$A^{RPG}W = \vec{F},$$

where $\vec{F}_i = (f, \psi_i)$, and

$$A_{i,j}^{RPG} = \tilde{B}(\phi_j, \psi_i).$$

We derive the A matrix coefficients using quadrature rules, and note again that the stiffness matrix is fully occupied. We note that the size of the matrix is $n \times N$ which is not square and we obtain the solution using least squares. An assumption is made in [10] that the RNN basis functions are linearly independent which is most likely but the rank of matrix degrades for N large.

As for the error analysis, first as B and $\tilde{B}$ are equivalent, we have a sort of orthogonality condition

$$\tilde{B}(u - u_{RPG}, v_h) = 0, \quad \forall v_h \in X_h. \quad (11)$$

The main result pertaining to the error analysis is as follows.

Lemma 1 (Céa Inequality). Let u satisfy (5) and u^{RPG} satisfy (10). Then we have

$$\|u - u_{RPG}\|_{H^{a/2}} \leq (1 + C_c/C_o) \inf_{v_h \in V_h} \|u - v_h\|_{H^{a/2}}. \quad (12)$$

Proof. We follow the same manner of proof as in [27]. From the coercivity equation, we have that.

$$C_o \|v_h - u_{RPG}\|_{H^{a/2}}^2 \leq \tilde{B}(v_h - u_{RPG}, v_h - u_{RPG}).$$

Next, adding and subtracting and using the orthogonality condition (11), we have

$$\begin{aligned} C_o \|v_h - u_{RPG}\|_{H^{a/2}}^2 &\leq \tilde{B}(v_h - u, v_h - u_{RPG}) + \tilde{B}(u - u_{RPG}, v_h - u_{RPG}) \\ &\leq C_c \|v_h - u\|_{H^{a/2}} \|v_h - u_{RPG}\|_{H^{a/2}}, \end{aligned}$$

from use of the continuity of the bilinear form. Finally, we obtain the stated result using the triangle inequality. $\square$

We immediately have from Lemma 1 that

$$\|u - u_{RPG}\|_{H^{a/2}} \leq Ch^{r-a/2} \|u\|_{H^r}, \quad (13)$$

where r is the maximum regularity of the solution and that admitted by the piecewise polynomial approximation used ($r = 2$ for linears, $r = 3$ for quadratics for example). The result (13) is not that useful computationally as it is unknown at this time how to compute fractional derivatives or fractional Sobolev norms on the RNN space. One would expect some sort of increased convergence in the L^2 norm though for this formulation we lack an efficient regularity estimate as well as a Galerkin orthogonality condition for the adjoint problem. Likewise, we would expect that the H^1 seminorm would converge at a rate of $r - 1$, where r represents the regularity of the underlying solution, with a maximum value of 2 when V_h is the set of piecewise linear polynomials.

Table 1

L^2 and H^1 convergence of the RPG simulation with respect to the grid parameter h in the test basis. Results are given for both $\alpha = 1.6$ and $\alpha = 1.75$.

h	$\ u - u_{RPG}\ _{L^2}$ ($\alpha = 1.6$)	$ u - u_{RPG} _{H^1}$	$\ u - u_{RPG}\ _{L^2}$ ($\alpha = 1.75$)	$ u - u_{RPG} _{H^1}$
1/4	1.306347e-3	1.122043e-1	9.210520e-3	9.887873e-2
1/8	1.420092e-4	1.391541e-3	1.405852e-4	1.328409e-3
1/16	2.145690e-5	2.191822e-4	3.983994e-5	2.848880e-4
1/32	8.157715e-6	1.544972e-4	1.678623e-5	1.975886e-4

Table 2

L^2 and H^1 convergence of the RPG simulation with respect to the grid parameter h in the test basis. Results are given for both $\alpha = 1.6$ and $\alpha = 1.75$.

h	$\ u - u_{RPG}\ _{L^2}$ ($\alpha = 1.6$)	$ u - u_{RPG} _{H^1}$	$\ u - u_{RPG}\ _{L^2}$ ($\alpha = 1.75$)	$ u - u_{RPG} _{H^1}$
1/4	5.92226e-3	1.318315e-1	2.377269e-3	4.266707e-2
1/8	1.892373e-3	1.018560e-1	6.590706e-4	3.512542e-2
1/16	7.189206e-4	8.443442e-2	2.202371e-2	2.586727e-2
1/32	3.021352e-4	7.378157e-2	8.142348e-5	2.021388e-2

5. Computational experiments

In this section, we give the results of six computational experiments of the RPG method. We give for each of the left-side, right-sided, and two-sided fractional diffusion equations an example where the true solution is fully regular and a second example where the true solution lacks regularity. We choose a neural network basis with $N = 4/h$ ($h = 1/4$ has a network with 16 interior nodes, $h = 1/8$ has a network with 32 interior nodes, etc.). We initialize the internal nodes using Matlab's built-in rand command all starting with the same kernel. We ensure that for each layer, the randomly chosen weights have an average value of zero. Experiments were carried out on a MacBook Air using Matlab R2022a.

Experiment 1 (Left-Sided Fractional Operator; Regular Solution). For the first computational experiment we implement the RPG method for the left side boundary value problem

$$\begin{aligned} \mathbf{D}^\alpha u &= f, \quad \text{on } 0 \leq x \leq 1, \\ u(0) &= u(1) = 0. \end{aligned} \quad (14)$$

We choose as the exact solution $u = x - x^4$ and have that the right hand side f equals to

$$f = -\frac{x^{1-\alpha}}{\Gamma(2-\alpha)} + \frac{\Gamma(5)x^{4-\alpha}}{\Gamma(5-\alpha)}.$$

In Table 1, we present the error analysis for $\alpha = 1.6, 1.75$. We see that convergence in the error is obtained.

Experiment 2 (Left-Sided Fractional Operator, Irregular Solution). For the second computational experiment we implement the RPG method for the left side boundary value problem with irregular solution

$$\begin{aligned} \mathbf{D}^\alpha u &= 1 - x, \quad \text{on } 0 \leq x \leq 1, \\ u(0) &= u(1) = 0. \end{aligned} \quad (15)$$

For this experiment, we can calculate the exact solution as

$$u = -\frac{x^\alpha - x^{\alpha-1}}{\Gamma(\alpha+1)} + \frac{x^{\alpha-1} - x^{\alpha-1}}{\Gamma(\alpha+2)}.$$

In Table 2, we present the error analysis for $\alpha = 1.6, 1.75$. We see that convergence in the error is obtained.

Experiment 3 (Right-Sided Fractional Operator; Regular Solution). For the third computational experiment we implement the RPG method for the right side fractional boundary value problem

$$\begin{aligned} \mathbf{D}^{\alpha*} u &= f, \quad \text{on } 0 \leq x \leq 1, \\ u(0) &= u(1) = 0. \end{aligned} \quad (16)$$

We choose as the exact solution $u = x - x^4$ and have that the right hand side f equals to

$$f = -\frac{3(1-x)^{1-\alpha}}{\Gamma(2-\alpha)} + \frac{6\Gamma(3)(1-x)^{2-\alpha}}{\Gamma(3-\alpha)} - \frac{4\Gamma(4)(1-x)^{3-\alpha}}{\Gamma(4-\alpha)} + \frac{\Gamma(5)(1-x)^{4-\alpha}}{\Gamma(5-\alpha)}.$$

In Table 3, we present the error analysis for $\alpha = 1.6, 1.75$. We see that convergence in the error is obtained.

Table 3

L^2 and H^1 convergence of the RPG simulation with respect to the grid parameter h in the test basis. Results are given for both $\alpha = 1.6$ and $\alpha = 1.75$.

h	$\ u - u_{RPG}\ _{L^2}$ ($\alpha = 1.6$)	$ u - u_{RPG} _{H^1}$	$\ u - u_{RPG}\ _{L^2}$ ($\alpha = 1.75$)	$ u - u_{RPG} _{H^1}$
1/4	3.835423e-3	3.575958e-2	2.722535e-3	3.232531e-2
1/8	2.953784e-4	3.992087e-3	1.576423e-4	2.837803e-3
1/16	5.170253e-5	1.11146e-3	3.577693e-5	8.549264e-4
1/32	2.929248e-5	9.510903e-4	1.815504e-5	7.210471e-4

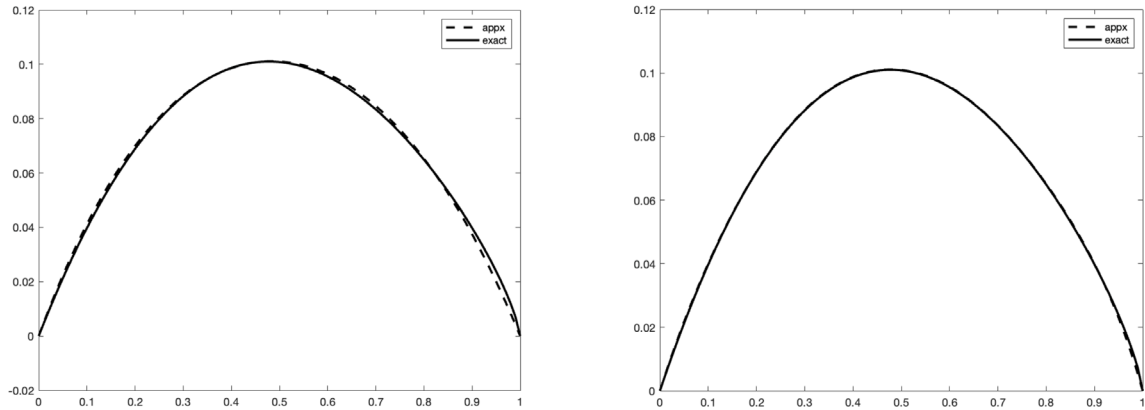


Fig. 3. Graphs of the approximate vs. exact solution for Experiment 4 with $\alpha = 1.75$. Left: $h = 1/4$ Right: $h = 1/8$.

Table 4

L^2 and H^1 convergence of the RPG simulation with respect to the grid parameter h in the test basis. Results are given for both $\alpha = 1.6$ and $\alpha = 1.75$.

h	$\ u - u_{RPG}\ _{L^2}$ ($\alpha = 1.6$)	$ u - u_{RPG} _{H^1}$	$\ u - u_{RPG}\ _{L^2}$ ($\alpha = 1.75$)	$ u - u_{RPG} _{H^1}$
1/4	4.790360e-3	8.271502e-2	1.372734e-3	2.882733e-2
1/8	1.704016e-3	6.455003e-2	9.574467e-4	2.616921e-2
1/16	6.872682e-4	5.650449e-2	4.092241e-4	1.906536e-2
1/32	1.103146e-3	5.867526e-2	2.720878e-4	1.738630e-2

Experiment 4 (Right-Sided Fractional Operator; Irregular Solution). For the fourth computational experiment we implement the RPG method for the right side fractional operator boundary value problem

$$\begin{aligned} \mathbf{D}^{\alpha*} u &= 1 - x, \quad \text{on } 0 \leq x \leq 1, \\ u(0) &= u(1) = 0. \end{aligned} \quad (17)$$

For this experiment, we can calculate the exact solution as

$$u = -\frac{(1-x)^{\alpha+1}}{\Gamma(\alpha+2)} + \frac{(1-x)^{\alpha-1}}{\Gamma(\alpha+2)}.$$

In Table 4, we present the error analysis for $\alpha = 1.6, 1.75$. We see that convergence in the error is obtained. In Fig. 3, we display a visual representation of the results when $\alpha = 1.75$ and $h = 1/4$ and $h = 1/8$, respectively.

Experiment 5 (Two-Sided Fractional Operator; Regular Solution). For the fifth computational experiment we implement the RPG method for the two sided fractional boundary value problem

$$\begin{aligned} \frac{1}{2} \mathbf{D}^{\alpha*} u + \frac{1}{2} \mathbf{D}^{\alpha*} u &= f, \quad \text{on } 0 \leq x \leq 1, \\ u(0) &= u(1) = 0. \end{aligned} \quad (18)$$

We choose as the exact solution $u = x - x^4$ and have that the right hand side f equals to a linear combination of the right hand side functions in Experiments 1 and 3, respectively. In Table 5, we present the error analysis for $\alpha = 1.6, 1.75$. We see that convergence in the error is obtained.

Table 5

L^2 and H^1 convergence of the RPG simulation with respect to the grid parameter h in the test basis. Results are given for both $\alpha = 1.6$ and $\alpha = 1.75$.

h	$\ u - u_{RPG}\ _{L^2}$ ($\alpha = 1.6$)	$ u - u_{RPG} _{H^1}$	$\ u - u_{RPG}\ _{L^2}$ ($\alpha = 1.75$)	$ u - u_{RPG} _{H^1}$
1/4	8.684162e-3	1.155501e-1	8.685702e-3	1.156558e-1
1/8	8.403413e-5	1.741682e-3	9.9344416e-5	1.788102e-3
1/16	3.142944e-5	3.89950e-4	4.1790321e-5	4.430658e-4
1/32	1.417520e-5	2.995756e-4	1.968816e-5	3.366034e-4

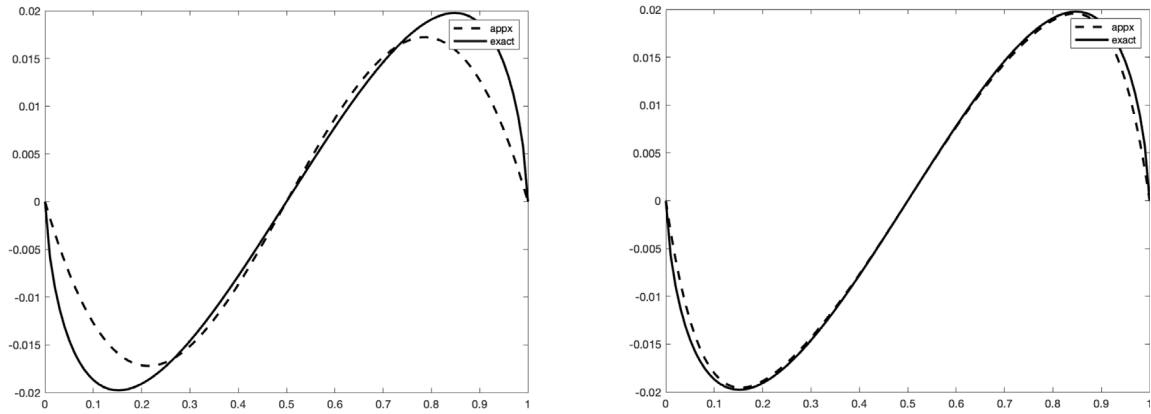


Fig. 4. Graphs of the approximate vs. exact solution for Experiment 6 with $\alpha = 1.75$. Left: $h = 1/4$ Right: $h = 1/8$.

Table 6

L^2 and H^1 convergence of the RPG simulation with respect to the grid parameter h in the test basis. Results are given for both $\alpha = 1.6$ and $\alpha = 1.75$.

h	$\ u - u_{RPG}\ _{L^2}$ ($\alpha = 1.6$)	$ u - u_{RPG} _{H^1}$	$\ u - u_{RPG}\ _{L^2}$ ($\alpha = 1.75$)	$ u - u_{RPG} _{H^1}$
1/4	6.232288e-3	1.130677e-1	3.374054e-3	1.597321e-1
1/8	1.806167e-3	7.340110e-2	8.906882e-4	1.500465e-1
1/16	6.702908e-4	5.375673e-2	3.062353e-4	1.469114e-1
1/32	3.550659e-4	4.819583e-2	1.542004e-4	1.461755e-1

Experiment 6 (Two-Sided Fractional Operator; Irregular Solution). For the final computational experiment we consider a two-sided operator but choose a true solution and right hand side which lacks full regularity. We solve

$$\frac{1}{2} \mathbf{D}^{\alpha*} u + \frac{1}{2} \mathbf{D}^{\alpha*} u = f, \quad \text{on } 0 \leq x \leq 1, \quad (19)$$

$$u(0) = u(1) = 0.$$

$$\text{where } f = -\frac{x^{1-\alpha}}{2\Gamma(2-\alpha)} + \frac{(1-x)^{1-\alpha}}{2\Gamma(2-\alpha)}.$$

From [22], we have that $u = x - k(x)/k(1)$, where

$$p = 0.5, \alpha = 1.6, k(x) = \int_0^x s^{-1/5}(1-s)^{-1/5} ds = \frac{5}{4} x^{4/5} {}_2F_1\left(\frac{1}{5}, \frac{4}{5}; \frac{9}{5}; x\right),$$

$$p = 0.5, \alpha = 1.75, k(x) = \int_0^x s^{-1/8}(1-s)^{-1/8} ds = \frac{8}{7} x^{7/8} {}_2F_1\left(\frac{1}{8}, \frac{7}{8}; \frac{15}{8}; x\right).$$

In Table 6, we present the error analysis for $\alpha = 1.6, 1.75$. We see that convergence in the error is obtained. In Fig. 4, we display a visual representation of the results when $\alpha = 1.75$ and $h = 1/4$ and $h = 1/8$, respectively.

In conclusion, we see from the computational experiments that the RPG method gives accurate solutions which converge well for boundary value problems with regular solutions. As with the finite element case, problems with irregular solutions suffer a degradation in the experimental convergence rates. The RNN concept with least squares promises to improve the existing PINN models and can be incorporated in the current state-of-the-science approaches in a number of ways, including as an augmented loss function, initial guess, or using domain decomposition. This approach can be extended to problems in two or three spatial dimensions by using an RNN with multiple inputs, and the approach can be studied for nonlinear and time dependent fractional partial differential equations.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] Sirignano J, Spiliopoulos K. DGM: A deep learning algorithm for solving partial differential equations. *J Comput Phys* 2018;375:1339–64.
- [2] Cai S, Mao Z, Wang Z, Yin M, Karniadakis GE. Physics-informed neural networks (PINNs) for fluid mechanics: A review. *Acta Mech Sin* 2021;37(12):1727–38.
- [3] Howard AA, Murphy SH, Ahmed SE, Stinis P. Stacked networks improve physics-informed training: applications to neural networks and deep operator networks. *Found Data Sci* 2024. accepted & online early access.
- [4] Pang G, Lu L, Karniadakis GE. fPINNs: Fractional physics-informed neural networks. *SIAM J Sci Comput* 2019;41(4):2603–26.
- [5] He J, Liu Y, Li H. TDOR-MPINNs: Multi-output physics-informed neural networks based on time differential order reduction for solving coupled Klein–Gordon–Zakharov systems. *Results Appl Math* 2024;22:100462.
- [6] Liu W, Liu Y, Li H. Time difference physics-informed neural network for fractional water wave models. *Results Appl Math* 2023;17:100347.
- [7] Igel'nik B, Pao YH. Stochastic choice of basis functions in adaptive function approximation and the functional-link net. *IEEE Trans Neural Netw* 1995;6:1320–9.
- [8] Huang GB, Zhu QY, Siew CK. Extreme learning machine: theory and applications. *Neurocomputing* 2006;70:489–501.
- [9] Shang Y, Wang F, Sun J. Randomized neural network with Petrov–Galerkin methods for solving linear and nonlinear partial differential equations. *Commun Nonlinear Sci Numer Simul* 2023;127:107518.
- [10] Sun J, Dong S, Wang F. Local randomized neural networks with discontinuous Galerkin methods for partial differential equations. *J Comput Appl Math* 2024;115830.
- [11] Sun J, Wang F. Local randomized neural networks with discontinuous Galerkin methods for diffusive-viscous wave equation. *Comput Math Appl* 2024;154:128–37.
- [12] Sokolov IM, Klafter J, Blumen A. Fractional kinetics. *Phys Today* 2002;48–53.
- [13] Kirchner JW, Feng X, Neal C. Fractal streamchemistry and its implications for contaminant transport in catchments. *Nature* 2000;403:524–6.
- [14] Benson DA, Wheatcraft SW, Meerschaert MM. The fractional order governing equations of Lévy motion. *Water Resour Res* 2000;36:1413–23.
- [15] Cortis A, Berkowitz B. Anomalous transport in classical soil and sand columns. *Soil Sci Soc Am J* 2004;68:1539–48.
- [16] Bates P, Chmaj A. An integrodifferential model for phase transitions: Stationary solutions in higher space dimensions. *J Stat Phys* 1999;95:1119–39.
- [17] Fife P. Some nonclassical trends in parabolic and parabolic-like evolutions. In: *Trends in nonlinear analysis*. Springer; 2003, p. 153–91.
- [18] Carreras BA, Lynch VE, Zaslavsky GM. Anomalous diffusion and exit time distribution of particle tracers in plasma turbulence models. *Phys Plasmas* 2001;8(12):5096–103.
- [19] Podlubny I. *Fractional differential equations*. New York: Academic Press; 1999.
- [20] Meerschaert MM, Tadjeran C. Finite difference approximations for fractional advection–dispersion flow equations. *J Comput Appl Math* 2004;172(1):65–77.
- [21] Yuste SB, Acedo L. An explicit finite difference method and a new von Neumann-type stability analysis for fractional diffusion equations. *SIAM J Numer Anal* 2005;42:1862–74.
- [22] Ervin VJ, Heuer N, Roop JP. Regularity of the solution to 1-D fractional order diffusion equations. *Math Comp* 2018;87(313):2273–94.
- [23] Ervin VJ, Roop JP. Variational formulation for a stationary fractional advection dispersion equation. *Numer Methods Partial Differential Equations* 2006;22:558–76.
- [24] Montufar GF, Pascanu R, Cho K, Bengio Y. On the number of linear regions of deep neural networks. *Adv Neural Inf Process Syst* 2014;27.
- [25] Gonon L, Grigoryeva L, Ortega JP. Approximation bounds for random neural networks and reservoir systems. *Ann Appl Probab* 2023;33(1):28–69.
- [26] Roop JP. Computational aspects of FEM approximation of fractional advection dispersion equations on bounded domains in $\mathbb{R}^2$. *J Comput Appl Math* 2006;193(1):243–68.
- [27] Griffiths DF, Lorenz J. An analysis of the Petrov–Galerkin finite element method. *Comput Methods Appl Mech Engrg* 1978;14(1):39–64.